Phase 1 Baseline Report

Resource-Bounded CTF Agents — Phase 1 Baseline

Team: Steven Zhang, Benjamin Yu, Jackie Liu · **Option 7** · **Repo:** https://github.com/StrawberrysAdvanture/cse291-ctf-agent

Research question. How well does a *small, self-hosted* model (7B, 4-bit, on a single mid-range GPU) solve CTF challenges under a fixed step and token budget, and **what limits it** — model capability, the tool interface, lost intermediate findings, or unproductive loops? Phase 1 establishes a reproducible baseline, measures its execution behavior, and uses the failure breakdown to justify a Phase 2 direction.

1. Setup

A tool-using agent with a **fixed** model, task instructions, tool set, and step budget, run on NYU CTF Bench.

- **Model:** Qwen2.5-7B-Instruct-AWQ (INT4 weights, FP16 compute/KV cache), self-hosted with vLLM 0.7.3 on one AWS g6.xlarge (NVIDIA L4, 24 GB). 32,768-token context, temperature 0.6, max_tokens 1024 per call.
- **Agent:** NYU CTF harness. Tools: `run_command`, `create_file`, `decompile_function`, `disassemble_function` (Ghidra), `check_flag`, `give_up`. The model has no native tool-calling; tool calls are emitted and parsed as XML inside a `<function_calls>…</function_calls>` block.
- **Budget:** 20 model calls and 900 s wall-clock per attempt.
- **Isolation:** the agent container runs on an internal Docker network (`--internal`, no internet); only the challenge server is reachable. Answer keys are never mounted or shown.
- **Workloads: 10 development + 20 held-out** challenges across 6 categories (crypto, forensics, misc, pwn, rev, web), chosen for low point value and buildable environments. The development set is used to design/instrument the method; the held-out set is used only to assess it. A 3-challenge pilot validated the pipeline (excluded from results).

Reproducibility. Model, runtime (vLLM 0.7.3), and dataset (NYU CTF Bench **v20250206**) are pinned; code is pinned at commit `0ccb3f8`. Each attempt runs in a **fresh container** (`docker compose … --force-recreate`), so no state carries between attempts. Each challenge gets **3 attempts** with deterministic seeds (`seed = 1000 + attempt_index`); temperature 0.6 makes runs stochastic, so we report per-attempt variability. Excluded challenges and reasons are listed in `experiments/phase1/challenges.yaml`; **no failed run or timeout is silently dropped** — every attempt has a crash-safe log and a recorded `finish_reason`. One-command reproduction: `experiments/phase1/run_batch.py --set {dev,heldout} --attempts 3 --config qwen_phase1.yaml`.

2. Verification

Each attempt is re-scored **outside the harness** (`analysis/score.py`) against the answer key read from the dataset on the host — not from the agent's own verdict. A solve records **flag origin**: it counts as *earned* only if the correct flag appeared in a tool output before it was submitted. Every

attempt is also checked for **suspicious access** (dataset metadata, `challenge.json`, the Docker socket, the network).

- **1 of 1** held-out solve was earned from tool output (`2023q-rev-baby_s_third`); **0** possibly-memorized solves.
- **0** attempts accessed dataset metadata, solutions, or the network (`network_internal: true` everywhere).
- Independent re-score **agreed with the harness on all 90 attempts** (0 discordant).

3. Results and measurements

Set	Attempts	Solved (att.)	Challenges ≥1	Mean rounds	Mean wall (s)	Mean tokens/att.	Peak context (tok)
Development	30	0 (0.0%)	0/10	18.2	586	189.7k	17,568
Held-out	60	**1 (1.7%)**	**1/20**	18.4	544	169.4k	16,294
Pilot	3	1 (33%)	1/3	14.0	284	131.2k	11,572

- **Solve rate:** held-out **1.7% / 1-of-20 challenges** (`baby_s_third`, rev); development **0%**. Per category, every cell is 0 except **held-out rev 1/4** (`by_category.csv`).
- **Variability (stochastic, 3 attempts):** the one solve was **not reliable** — `baby_s_third` solved on attempt 0 (in 7 rounds) but hit `max_rounds` on attempts 1 and 2. No other challenge solved on any attempt.
- **Time & tokens:** ~540–590 s and ~170–190k cumulative tokens per attempt (sum over ≤20 calls); peak single-call context ~16–18k of the 32k budget.
- **Tool-call latency:** median **0.05 s**, p90 0.11 s, max **30 s** (Ghidra decompile/ disassemble). Tool latency is negligible; **model latency dominates** wall time.
- **Invalid commands:** high — **67%** of dev and **52%** of held-out tool calls returned an error or non-zero exit (1,066 of 1,849 total).
- **Timeouts: 0 attempt-level** timeouts (none hit the 900 s wall as finish reason); **60 command-level** timeouts, all held-out (47 crypto, 13 pwn) — interactive binaries blocking on stdin until the `run_command` limit.
- **Context growth:** climbs with accumulated tool output (Fig 2); 11 attempts (4 dev, 7 held-out) overflowed into `context_length`.
- **Cost:** ~**$9.4** total GPU (~11 g6.xlarge-hours @ ~$0.80/hr; pilot + dev + held-out + uniform dev re-run), well under the $20/day, $200 total cap (`experiments/phase1/aws_usage.md`, reconciled against CloudTrail).

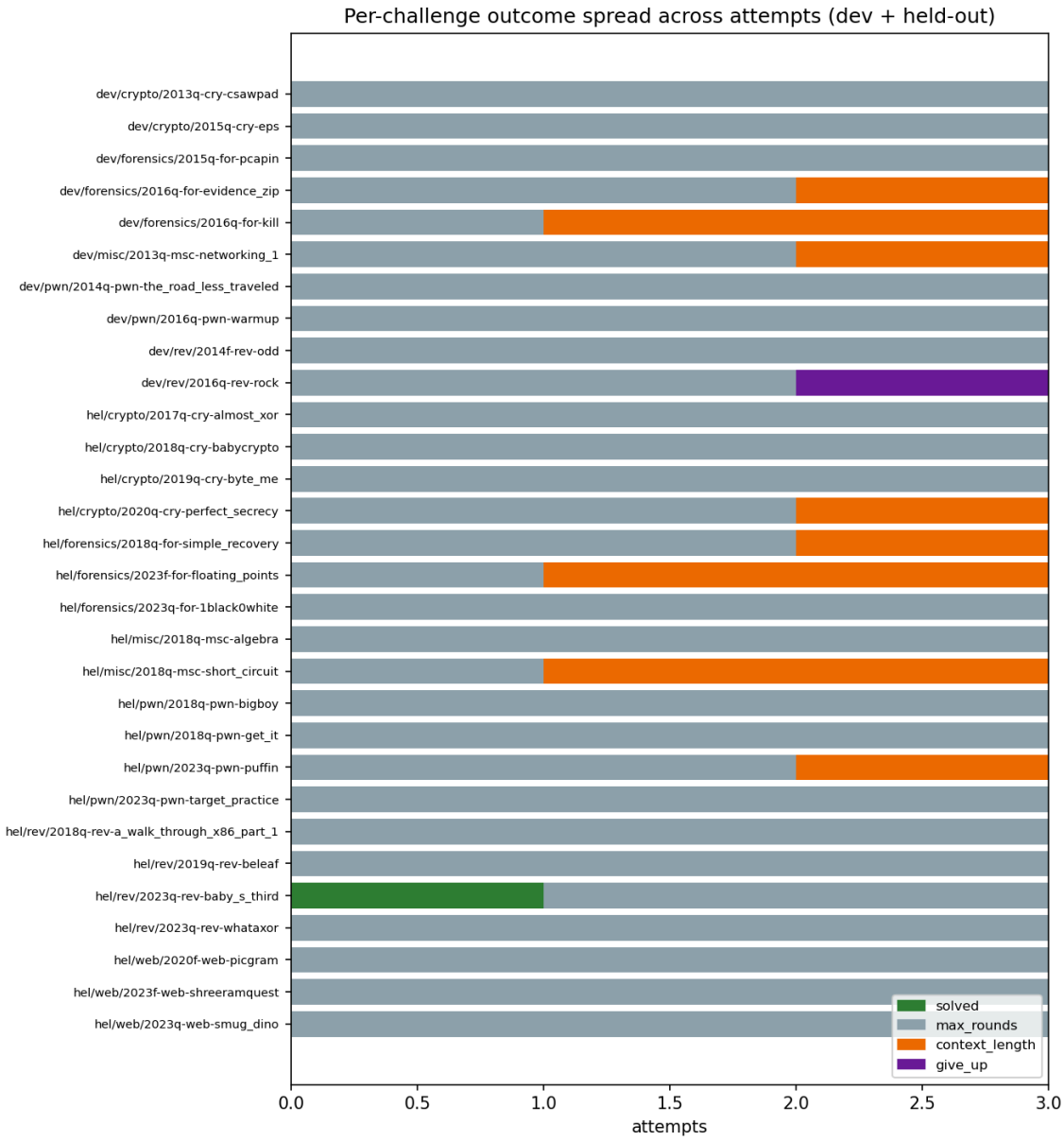


Fig 1. Outcome spread per challenge over 3 attempts — almost all end in `max_rounds` or `context_length`; one solve, on one of three attempts.

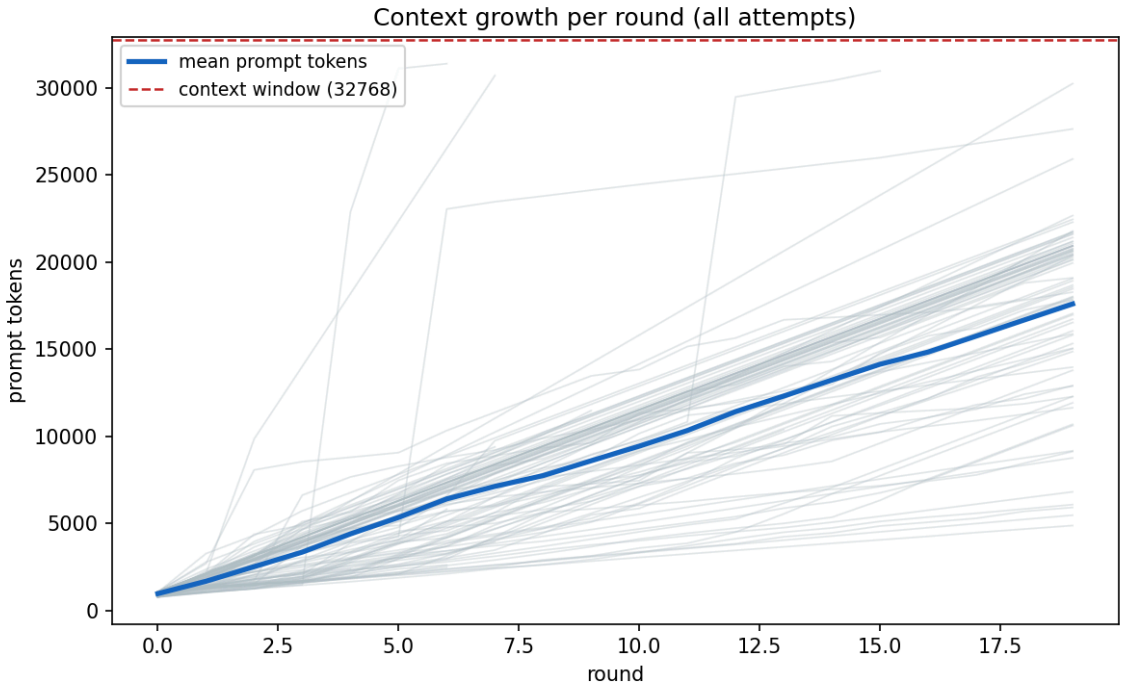


Fig 2. Prompt tokens vs. round; a few attempts spike into the 32k ceiling (`context_length`).

4. Failure analysis

89 of 90 attempts failed (finish reasons: 77 `max_rounds`, 11 `context_length`, 1 `give_up`). Hand-inspecting traces, we sort the causes into the four prescribed categories (combined dev + held-out, 1,647 model rounds, 1,849 tool calls):

1. **Poor tool interface — the largest *mechanical* loss. 57%** of rounds (938/1,647) produced **no executed tool call**. Many are **format slips**: the model emits a well-formed `<invoke>…</invoke>` *without* the `<function_calls>` wrapper the parser requires, so the harness drops it (191 such rounds in a seed-matched 32k run); an unwrapped invoke also has no stop sequence, so **~50%** of replies run to the 1,024-token cap (816/1,647 truncated). The model also calls programs (`cat`, `python3`, `sleuthkit`, `print_flag`) as if they were tools (~23 rejected), and the 52–67% invalid-command rate reflects malformed or wrong commands.
2. **Unproductive execution loops. ~1,160 ineffective repeats** (424 dev + 736 held-out): the agent re-issues commands whose output it has already seen, spending rounds without new information — the dominant pattern in `max_rounds` traces.
3. **Lost intermediate findings.** Tied to (2): with no memory of verified outputs, earlier results fall out of attention and the agent re-derives or re-runs them; large outputs are the same ones that cause the 11 `context_length` overflows (a *symptom* of unmanaged history, not steady growth — peaks averaged only ~16–18k of 32k).
4. **Missing model knowledge — the binding constraint.** Even when rounds execute cleanly, a 7B model rarely produces the multi-step exploitation crypto/pwn/rev/forensics need. `check_flag` was called 213 times, so the agent *attempts* solutions, but capability — not budget — is the ceiling.

Cross-validation (teammate branch `aws-gpu-runner`). An independent harness-hardening effort ran a **paired A/B over the full 57-challenge test set**: a clean baseline **0/57** and a version with *every* mechanical fix applied (lenient parsing recovering the unwrapped invokes, per-output caps, repeat detection, format nudges) **also 0/57** (exact McNemar p = 1.0), despite driving no-tool replies from 56% to **0%** and recovering 354 dropped calls. This independently confirms that category (4) is binding: **interface fixes alone do not produce solves.** Both efforts also converged independently on the same two low-level fixes (treat context overflow as `context_length`; cap oversized tool outputs), which we take as corroboration.

5. Phase 2 direction (justified by §4)

Category (4) says raw solve rate is capability-bound, but (1)–(3) show **half the budget is wasted before capability is even tested**. Phase 2 targets that waste and measures whether recovering it moves the needle, with controlled comparisons.

- **Primary: output budgeting / tool-feedback compaction** — cap each tool result (head + tail + a re-read note) so a single large output cannot overflow context. **Preliminary (branch `phase2-outputcap`):** a seed-matched held-out A/B (8k-char cap, native 32k) **cut `context_length` failures 7 → 3** with **no lost solves** (still 1/20, `baby_s_third` preserved, earned); dev blowouts fell 3 → 1. **No blowout converted to a solve** — consistent with §4.
- **Secondary: lenient tool-call parsing** to recover the ~57% dead rounds from format slips, making the budget productive and the baseline fairer.
- **Open question:** whether any *capability*-side aid (verified-findings memory to cut the ~1,160 repeats, or targeted tool/prompt scaffolding) helps where interface fixes alone did not.

All Phase 2 methods will be compared against the simple agent **and** fixed retries at equal total budget, with an ablation, on held-out challenges only.

Appendix: timeline and milestones

- **Phase 1 (done, Nov 4):** harness instrumentation, network isolation, independent scoring, reproducible dev + held-out runs (3 attempts each, fixed commit), execution measurements, and failure analysis. Code, configs, results, and logs: see the repository above.
- **Nov 4:** output-capping experiment complete (dev + held-out); results in §5.
- **[~Nov 20]:** Phase 2 mechanism finalized + ablation + fixed-retries baseline on held-out.
- **[~Dec 5]:** full held-out evaluation, variability runs, write-up.
- **Dec 10:** Phase 2 code and final report.